Contents

Abstract **3**
1. INTRODUCTION and Background **4**
1.1 Product (Problem Statement) 4
1.2 Background 4
1.3 Scope 4
1.4 Objectives 5
1.5 Challenges 6
1.6 Learning Outcomes 6
1.7 Nature of End Product 6
1.8 Completeness Criteria 7
1.9 Business Goals 7
1.10 Literature Review 7
1.11 Document Conventions 8
2. Overall Description **8**
2.1 Product Features 8
2.2 User Classes and Characteristics 8
2.3 Operating Environment 9
2.4 Design and Implementation Constraints 9
2.5 Assumptions and Dependencies 10
3. Functional Requirements **10**
3.1 Detailed Use Cases 10
Part 1: Passenger Use Cases (1-14) 10
3.1.1 Passenger Registration 10
3.1.2 Register with CNIC OCR 11
3.1.3 Passenger Login 11
3. 1.4 Update Profile & Emergency Contacts 12
3. 1.5 Pink Pass Enrollment (Identity Scan) 13
3. 1.6 Pink Pass Liveness Check 13
3. 1.7 Request Ride (Standard/Pink Pass) 14
3. 1.8 AI Price Prediction & Estimation 15
3. 1.9 Track Ride & Real-time ETA 15
3. 1.10 In-App Communication (Chat) 16

3. 1.11 Respond to 'Are you okay?' Safety Prompt 16
3. 1.12 Trigger Manual SOS 17
3. 1.13 Ride Feedback & Sentiment Submission 18
3. 1.14 View Personal Ride History 18
3.15 Driver Registration 19
3. 1.16 Driver PWA Login 20
3. 1.17 Toggle Availability (Online/Offline) 20
3. 1.18 Accept/Reject Ride 21
3. 1.19 Navigate to Pickup/Destination 21
3. 1.20 Manage Ride Status 22
3. 1.21 View Earnings & Fairness Score 23
3. 1.22 Rate Passenger 23
3. 1.23 Admin Secure Login 24
3. 1.24 Real-time Live Monitoring Map 24
3. 1.25 Safety Sentinel Alert Management 25
3. 1.26 Route Deviation Monitoring (Auto) 26
3. 1.27 Manage User Verification & Suspend 26
3. 1.28 Generate AI Demand Heatmaps 27
3. 1.29 Analyze Feedback Sentiment Analytics 27
3. 1.30 System Reports & Export 28
3.3 Requirements Analysis and Modeling 29
4. Nonfunctional Requirements 30
4.1 Performance Requirements 30
4.2 Safety Requirements 30
4.3 Security Requirements 30
4.4 Additional Software Quality Attributes 31
5. Other Requirements 31
7. References 35
Appendix A: Glossary 36

Abstract

SAFORA (Smart Ride) is an AI-powered ride-hailing platform designed to revolutionize passenger safety in Pakistan's transportation sector. Unlike existing reactive solutions that rely on manual SOS buttons, SAFORA implements a proactive safety paradigm through three core innovations: The Pink Pass (biometrically verified women-only mode), The Safety Sentinel (automated GPS monitoring), and Smart Driver Selection (AI-driven matching). This document outlines the software requirements for the initial phase of the SAFORA project, detailing the architectural, functional, and non-functional specifications necessary for development.

The system employs a hybrid microservice architecture, combining a MERN stack (MongoDB, Express, React, Node.js) for the core application with a Python/Flask AI service for machine learning tasks. Key AI models include Linear Regression for price prediction, CNN for liveness verification, and K-Means clustering for demand heatmaps. By integrating these advanced technologies, SAFORA aims to address critical gaps in current ride-hailing services, specifically focusing on the safety of female passengers and the equitable distribution of rides among drivers.

Furthermore, this SRS document provides a comprehensive breakdown of the system's use cases, including user registration, ride management, and emergency response protocols. It establishes firmly the constraints, assumptions, and dependencies that will guide the development process. The ultimate goal of this specification is to serve as a blueprint for a robust, scalable, and secure platform that restores trust in public transportation through technological innovation.

1. Introduction and Background

1.1 Product (Problem Statement)

Current ride-hailing services in Pakistan, such as Uber, Careem, InDrive, and Yango, suffer from three critical safety and operational deficiencies that severely impact user trust and reliability. First, these platforms rely primarily on reactive safety measures, such as manual SOS buttons, which are often ineffective in acute emergencies where a victim may be physically unable to access their phone. Second, gender verification mechanisms are porous; women-only modes rely on self-declared gender data, allowing unauthorized users to exploit the feature, thereby compromising the safety of female passengers who specifically seek secure transport options.

Third, existing driver allocation algorithms prioritize proximity above all else, leading to a "winner-takes-all" dynamic where top-rated drivers monopolize requests while others sit idle. This lack of fairness affects driver retention and service quality. Furthermore, the absence of transparent pricing models often leads to disputes between passengers and drivers. SAFORA addresses these systemic failures by introducing a proactive safety paradigm, rigorous biometric verification, and a fairness-aware distribution algorithm.

1.2 Background

The ride-hailing domain in Pakistan is a rapidly expanding market, driven by urbanization and the lack of reliable public transport infrastructure. However, this growth faces significant headwinds due to safety concerns. High-profile incidents of route deviation, harassment, and theft have eroded public confidence. While international players have established a foothold, their global standard operating procedures often fail to account for the specific security challenges of the local environment. SAFORA is conceived against this backdrop as a locally engineered solution that leverages advanced Artificial Intelligence to provide a layer of security that acts before an incident occurs, rather than merely reporting it afterwards.

1.3 Scope

The scope of the SAFORA project encompasses the end-to-end development of a comprehensive ride-hailing ecosystem. This includes:

1. A cross-platform mobile application for Passengers (Android/iOS) featuring real-time booking, tracking, and the "Pink Pass" verification interface.
2. A dedicated mobile application for Drivers that manages ride requests, navigation, and earnings.
3. A centralized Backend Server responsible for orchestration, user management, and secure data handling.
4. A specialized AI Microservice that hosts the machine learning models for liveness detection, price prediction, and route monitoring.
5. An Administrative Dashboard for real-time fleet supervision and incident response.

Key features within scope include the Pink Pass biometric verification system, the Safety Sentinel automated route monitoring engine, and the smart driver matching algorithm. Explicitly out of scope for this initial phase are multi-city operations (launch is limited to Lahore), integration with third-party digital wallets (cash-only initially), and carpooling features.

1.4 Objectives

1. Enhance Passenger Safety: Implementation of automated route deviation detection capable of identifying anomalies greater than 500 meters within 30 seconds, reducing emergency response latency significantly compared to manual reporting.

2. Ensure Gender Verification Integrity: Achieve greater than 98% accuracy in liveness detection to effectively prevent identity spoofing, ensuring that the "Pink Pass" mode remains exclusive to verified female users.

3. Optimize Driver Assignment: Deploy a weighted matching algorithm that reduces average passenger wait times to under 3 minutes while ensuring that 80% of available drivers receive at least one ride per hour during peak periods.

4. Provide Transparent Pricing: Utilize linear regression models to predict fares with a variance of less than PKR 30, providing users with a detailed cost breakdown before they confirm their booking.

5. Enable Proactive Safety Monitoring: Establish a real-time monitoring loop that automatically triggers alerts to administrators and emergency contacts without requiring user intervention, shifting the safety paradigm from reactive to proactive.

1.5 Challenges

1. Real-time GPS Accuracy: Ensuring reliable deviation detection is technically challenging due to GPS drift and signal loss in dense urban environments. The system must filter noise from actual anomalies to prevent false alarms.

2. AI Model Latency: Running complex computer vision models for liveness detection on mobile devices or transmitting video data to a server introduces latency. Optimizing these models for near-instantaneous response is critical for user experience.

3. Network Instability: Pakistan's mobile network infrastructure can be inconsistent. The application must be robust enough to handle intermittent connectivity without losing critical state data or failing to transmit safety alerts.

4. Privacy versus Security: Balancing the need for intrusive biometric verification and continuous location tracking with user privacy rights and data protection regulations requires careful architectural planning and encryption implementation.

1.6 Learning Outcomes

The development of SAFORA offers a diverse range of technical and professional learning outcomes. Technically, the team will gain deep expertise in microservices architecture, integrating heterogeneous systems (Node.js and Python), and deploying machine learning models in a production environment. We will master real-time geospatial data processing using technologies like Socket.io and Turf.js. Professionally, the project fosters skills in agile project management, requirements engineering, and the implementation of industry-standard security practices such as JWT authentication and AES-256 data encryption.

1.7 Nature of End Product

The final deliverable is a production-ready software suite consisting of two mobile applications (Passenger and Driver) and a web-based administrative console. These interfaces are powered by a robust backend infrastructure deployed on cloud servers. The end product will support real-world ride-hailing workflows, from account creation and identity verification to ride completion and payment. It is not merely a prototype but a functional system capable of handling concurrent users, processing real-time payments, and executing safety protocols in a live environment.

1.8 Completeness Criteria

The project will be deemed complete upon the satisfaction of the following rigorous criteria:

1. **Functional Integrity:** Use cases for all three user roles (Passenger, Driver, Admin) must be fully operational without critical bugs.
2. **Safety Validation:** The Safety Sentinel must successfully trigger alerts in 100% of tested simulated route deviations greater than the threshold.
3. **Biometric Accuracy:** The Pink Pass system must reject at least 95% of spoofing attempts (photos/videos) while accepting valid live users.
4. **Performance Benchmarks:** The system must support 100 concurrent uniform users with API response times remaining under 500ms.
5. **Documentation:** Comprehensive technical manuals, user guides, and API documentation must be delivered.

1.9 Business Goals

From a business perspective, SAFORA aims to disrupt the local market by capturing 15% of the female ride-hailing demographic in Lahore within the first year of operation. The platform targets a 40% reduction in safety-related support tickets compared to industry averages, translating to lower operational costs and higher brand loyalty. Furthermore, the verified safety features are positioned to establish B2B partnerships with corporate entities seeking safe travel options for their female workforce, creating a sustainable revenue stream beyond individual fares.

1.10 Literature Review

A comprehensive review of existing literature and competitive platforms reveals significant gaps. While Uber and Careem dominate the market, their safety features are largely reactive. Research by Zhang et al. (2021) in the "Journal of Transportation Safety" demonstrates that machine learning models can predict driver anomalies with 92% accuracy using GPS data, a capability currently absent in major local apps. Similarly, Patel et al. (2020) in "IEEE Transactions on Biometrics" confirm the efficacy of CNN-based liveness detection in preventing identity fraud, validating SAFORA's architectural choice for the Pink Pass. Comparison with "Her Way" in South Africa shows that while gender-segregated services exist, SAFORA's addition of automated biometric verification represents a novel technological advancement in this niche.

1.11 Document Conventions

This document adheres to strict typographical conventions to ensure clarity. Italicized terms denote external systems or third-party APIs (e.g., **Google Maps API, Twilio**).

Bold text is used to highlight key terms, identifiers (e.g., **UC-01**), and critical requirements. The document follows the IEEE 830-1998 standard for structure. Tense usage implies binding requirements ("shall") versus non-binding recommendations ("should"). All currency figures are cited in Pakistani Rupees (PKR) unless otherwise noted.

2. Overall Description

2.1 Product Features

SAFORA provides a comprehensive suite of features designed for safety and efficiency. **1. Pink Pass:** A specialized mode for female passengers that enforces strict biometric checks to ensure that only verified women can request or accept these rides.

2. Safety Sentinel: An always-on background service that monitors GPS coordinates against the planned route, automatically triggering alerts for unexplained deviations or stops.

3. Smart Driver Selection: A multifaceted matching engine that balances proximity with driver ratings and fairness metrics to optimize fleet utilization.

4. Dynamic Price Prediction: A transparency tool that provides users with a fair fare estimate based on distance, time, and current demand, reducing negotiation friction.

5. Admin Command Center: A dashboard providing real-time visibility into the entire network, including live ride tracking, heatmap visualizations of demand, and instant access to safety alerts.

2.2 User Classes and Characteristics

Passenger: The primary consumer of the service. Includes the general public, aged 18-65, who require reliable transport. They are assumed to have basic smartphone literacy. A critical subset is the **Pink Pass User**, female passengers who prioritize safety and are willing to undergo additional verification steps.

Driver: Service providers who own their vehicles. They are motivated by income stability and fair treatment. They possess moderate technical skills, capable of using navigation apps. Key requirements include valid licensure and vehicle registration.

Administrator: Internal staff responsible for platform health. They are technically proficient, capable of interpreting data analytics, and trained in emergency response protocols. They operate the dashboard to monitor safety compliance and manage user disputes.

2.3 Operating Environment

Client Layer: The mobile applications are built to run on Android devices (version 8.0 Oreo and above) and iOS devices (version 13.0 and above), ensuring broad compatibility with the local device market.

Server Layer: The backend infrastructure is hosted on cloud services (AWS/DigitalOcean/Heroku), utilizing Node.js for the API Gateway and Python for the AI Microservice.

Data Layer: MongoDB v6.0+ is used for flexible document storage, hosted on MongoDB Atlas for high availability.

Network: The system requires a continuous internet connection, functioning effectively on 4G/LTE networks and maintaining stability on 3G connections common in developing areas.

2.4 Design and Implementation Constraints

Regulatory: The system must strictly adhere to the draft Personal Data Protection Bill of Pakistan, particularly regarding the encryption of biometric data and CNIC information. SMS notifications must comply with PTA guidelines.

Technical: The solution is constrained by the need to use Google Maps API, which has associated costs; the architecture must optimize API calls to stay within budget. Mobile battery usage for GPS tracking must be optimized to not exceed 15% drain per hour.

Economic: As an FYP, the budget is limited. The system design prioritizes free-tier services and open-source technologies (MERN stack, Flask) to maintain financial viability during the development phase.

2.5 Assumptions and Dependencies

Assumptions: It is assumed that end-users possess smartphones with functional GPS sensors and cameras. We assume that the GPS accuracy provided by consumer devices is within a 10-meter margin of error. It is also assumed that the cellular network will provide at least 90% uptime in the target operational area.

Dependencies: The system is critically dependent on the Google Maps API for routing and geocoding services. It relies on Twilio for the delivery of SMS alerts; any outage in these third-party services will impact core functionality. The AI models depend on the availability of sufficient training data to achieve the targeted accuracy rates.

3. Functional Requirements

All functional requirements are detailed below as formal Use Cases.

3.1 Detailed Use Cases

Part 1: Passenger Use Cases (1-14)

3.1.1 Passenger Registration

Identifier	UC-01
Purpose	Allows a new passenger to create a basic account profile.
Priority	High
Pre-conditions	App is installed; Smartphone has internet connectivity.
Post-conditions	Basic account record created in database.

Typical Course of Action

S#	Actor Action	System Response
1	Passenger selects 'Sign Up'	System displays registration form
2	Passenger enters Name, Email, and Phone Number	System validates email format
3	Passenger creates a secure password	System encrypts and stores password

4		System confirms account creation

Alternate Course of Action

S#	Actor Action	System Response
1	Invalid email format	System displays error message and prompts for correction

3.1.2 Register with CNIC OCR

Identifier	UC-02
Purpose	Verifies identity by extracting data from CNIC using AI OCR.
Priority	High
Pre-conditions	UC-01 in progress; Camera permissions granted.
Post-conditions	Identity verified; Account fully activated.

Typical Course of Action

S#	Actor Action	System Response
1	Passenger takes photo of CNIC (Front/Back)	System activates AI OCR module
2		System extracts CNIC number and Gender
3		System validates CNIC with government records
4	Passenger enters received OTP	System verifies phone number and activates account

Alternate Course of Action

S#	Actor Action	System Response
1	Blurred photo detected	System prompts user to retake the photo

3.1.3 Passenger Login

Identifier	UC-03

Purpose	Authentication via credentials for secure access.
Priority	High
Pre-conditions	Account exists and is verified.
Post-conditions	Access granted to passenger dashboard.

Typical Course of Action

S#	Actor Action	System Response
1	User enters Email/Phone and Password	System checks credentials against database
2		System generates 24-hour JWT token
3		System redirects to Home screen

Alternate Course of Action

S#	Actor Action	System Response
1	Incorrect credentials	System displays "Invalid Credentials" and locks after 5 attempts

3. 1.4 Update Profile & Emergency Contacts

Identifier	UC-04
Purpose	Manage personal details and safety contacts for the Safety Sentinel.
Priority	Medium
Pre-conditions	Logged in.
Post-conditions	Profile and safety contacts updated.

Typical Course of Action

S#	Actor Action	System Response
1	Passenger navigates to 'Profile Settings'	System displays current profile data
2	Passenger updates personal info or adds 5 Emergency Contacts	System validates inputs
3	Passenger selects 'Save'	System updates records in MongoDB

Alternate Course of Action

S#	Actor Action	System Response
1	Invalid phone number for contact	System highlights the error and prevents saving

3. 1.5 Pink Pass Enrollment (Identity Scan)

Identifier	UC-05
Purpose	Initial phase of women-only mode activation involving document check.
Priority	High
Pre-conditions	Logged in; Registered as Female via CNIC OCR.
Post-conditions	System ready for biometric verification.

Typical Course of Action

S#	Actor Action	System Response
1	Passenger selects 'Pink Pass' registration	System checks gender metadata
2	Passenger grants camera permissions	System prepares biometric liveness check

Alternate Course of Action

S#	Actor Action	System Response
1	Gender is not 'Female'	System restricts access to this feature

3. 1.6 Pink Pass Liveness Check

Identifier	UC-06
Purpose	AI-driven biometric check using face detection and blink detection.
Priority	High
Pre-conditions	UC-05 completed.
Post-conditions	Pink Pass activated for the user.

Typical Course of Action

S#	Actor Action	System Response

1	Passenger blinks/smiles at camera	System captures video frames
2		AI Service calculates EAR and liveness score
3		System activates Pink Pass status if score > 0.95

Alternate Course of Action

S#	Actor Action	System Response
1	Spoof detected (e.g., photo)	System denies verification and logs the attempt

3. 1.7 Request Ride (Standard/Pink Pass)

Identifier	UC-07
Purpose	Booking a ride after selecting destination and ride type.
Priority	High
Pre-conditions	Logged in; Location services enabled.
Post-conditions	Ride request broadcasted to matching drivers.

Typical Course of Action

S#	Actor Action	System Response
1	Passenger sets destination on map	System displays available car categories
2	Passenger selects 'Standard' or 'Pink Pass' (if eligible)	System calculates trip parameters
3	Passenger clicks 'Confirm Request'	System starts matching algorithm

Alternate Course of Action

S#	Actor Action	System Response
1	No drivers available	System notifies user to try again later

3. 1.8 AI Price Prediction & Estimation

Identifier	UC-08
Purpose	Calculates fair transparent pricing based on demand and traffic.
Priority	Medium
Pre-conditions	Destination set in UC-07.
Post-conditions	

Typical Course of Action

S#	Actor Action	System Response
1		System sends trip data to AI microservice
2		AI Model predicts price using Linear Regression
3		System factors in surge and traffic
4		System displays 'Estimated Fare' with breakdown

Alternate Course of Action

S#	Actor Action	System Response
1	AI Service offline	System uses fallback distance-based calculation

3. 1.9 Track Ride & Real-time ETA

Identifier	UC-09
Purpose	Live monitoring of the driver's approach and trip progress.
Priority	High
Pre-conditions	Ride accepted or active.
Post-conditions	Passenger views live progress on map.

Typical Course of Action

S#	Actor Action	System Response
1		System polls driver GPS every 5 seconds

2		System updates car icon and ETA on passenger map
3		System triggers 'Driver Arrived' notification when < 100m

Alternate Course of Action

S#	Actor Action	System Response
1	GPS signal lost	System displays "Poor connection" and shows last known location

3. 1.10 In-App Communication (Chat)

Identifier	UC-10
Purpose	Secure chat between passenger and driver without sharing numbers.
Priority	Medium
Pre-conditions	Ride assigned.
Post-conditions	Successful coordination between parties.

Typical Course of Action

S#	Actor Action	System Response
1	Passenger/Driver sends text message	System delivers via Socket.io
2		System issues push notification to recipient

Alternate Course of Action

S#	Actor Action	System Response
1	Network failure	System displays "Failed to send" with retry option

3. 1.11 Respond to 'Are you okay?' Safety Prompt

Identifier	UC-11
Purpose	Responding to automated system checks during anomalies.

Priority	High
Pre-conditions	Safety Sentinel detects deviation (UC-26).
Post-conditions	Alert either suppressed or escalated.

Typical Course of Action

S#	Actor Action	System Response
1		System detects deviation and triggers safety prompt
2	Passenger taps 'I am safe' within 30s	System logs response and resets deviation timer
3		System resumes background monitoring

Alternate Course of Action

S#	Actor Action	System Response
1	Passenger taps 'Help!' or timer expires	System triggers Emergency Protocol (UC-12)

3. 1.12 Trigger Manual SOS

Identifier	UC-12
Purpose	Manual emergency trigger in case of immediate threat.
Priority	High
Pre-conditions	Ride active.
Post-conditions	Emergency protocol activated.

Typical Course of Action

S#	Actor Action	System Response
1	Passenger taps red SOS button	System activates immediate emergency mode
2		System broadcasts location to Admin dashboard
3		System sends automated SMS to all emergency contacts

Alternate Course of Action

S#	Actor Action	System Response
1	Accidental trigger (taps 'Cancel' in 5s)	System logs the event but suppresses alerts

3. 1.13 Ride Feedback & Sentiment Submission

Identifier	UC-13
Purpose	Providing rating and textual feedback for AI analysis.
Priority	Medium
Pre-conditions	Ride completed.
Post-conditions	Sentiment data stored for platform quality control.

Typical Course of Action

S#	Actor Action	System Response
1	Passenger submits 1-5 star rating and comment	System saves rating to driver profile
2		AI Service analyzes text for sentiment score
3		System flags negative sentiment to Admin

Alternate Course of Action

S#	Actor Action	System Response
1	Feedback contains prohibited language	System censors/rejects feedback and notifies Admin

3. 1.14 View Personal Ride History

Identifier	UC-14
Purpose	Accessing previous trips and spending history.
Priority	Low
Pre-conditions	Logged in.
Post-conditions	History log displayed.

Typical Course of Action

S#	Actor Action	System Response
1	User navigates to 'Ride History'	System fetches records from MongoDB
2		System displays list of past trips with costs

Alternate Course of Action

S#	Actor Action	System Response
1	No history found	System displays "No rides found" message

Part 2: Driver Use Cases (15-22)

3.15 Driver Registration

Identifier	UC-15
Purpose	Sign-up process for drivers including documentation.
Priority	High
Pre-conditions	Valid license and vehicle documents available.
Post-conditions	Driver account created (Status: Pending Approval).

Typical Course of Action

S#	Actor Action	System Response
1	Driver enters personal info and car category	System displays document upload interface
2	Driver uploads License, CNIC, and Registration	System saves encrypted files to storage
3	Driver submits application	System creates 'Pending' account in database

Alternate Course of Action

S#	Actor Action	System Response
1	Expired documents uploaded	System flags expiration dates and warns user

3. 1.16 Driver PWA Login

Identifier	UC-16
Purpose	Secure access to the driver's Progressive Web App interface.
Priority	High
Pre-conditions	Account created and approved.
Post-conditions	Driver accesses management tools.

Typical Course of Action

S#	Actor Action	System Response
1	Driver enters credentials via PWA	System validates user and token
2		System launches Driver Dashboard

Alternate Course of Action

S#	Actor Action	System Response
1	Account not yet approved	System displays "Approval in Progress" status page

3. 1.17 Toggle Availability (Online/Offline)

Identifier	UC-17
Purpose	Managing whether the driver is ready to receive requests.
Priority	High
Pre-conditions	Logged in.
Post-conditions	Status updated in real-time server state.

Typical Course of Action

S#	Actor Action	System Response
1	Driver toggles 'Go Online'	System starts background GPS tracking
2		System adds driver to matching pool

Alternate Course of Action

S#	Actor Action	System Response

1	Low battery warning	System displays alert recommending charger connection

3. 1.18 Accept/Reject Ride

Identifier	UC-18
Purpose	Responding to incoming matched ride requests.
Priority	High
Pre-conditions	Driver online; Successfully matched by AI.
Post-conditions	Ride assigned to the driver.

Typical Course of Action

S#	Actor Action	System Response
1		System triggers ringing alert on Driver app
2	Driver accepts within 15s	System confirms the match
3		System sends driver details to passenger

Alternate Course of Action

S#	Actor Action	System Response
1	Driver rejects or timer expires	System removes driver from this match and re-runs AI matching

3. 1.19 Navigate to Pickup/Destination

Identifier	UC-19
Purpose	Using integrated Google Maps for turn-by-turn guidance.
Priority	High
Pre-conditions	Ride accepted.
Post-conditions	Efficient arrival at target location.

Typical Course of Action

S#	Actor Action	System Response
1	Driver taps 'Start Navigation'	System launches Google Maps SDK with route
2		System monitors distance progress

Alternate Course of Action

S#	Actor Action	System Response
1	Maps API quota exceeded	System provides static route/address as fallback

3. 1.20 Manage Ride Status

Identifier	UC-20
Purpose	Tracking the physical progression of the passenger journey.
Priority	High
Pre-conditions	Ride assigned.
Post-conditions	Ride state transitioned in backend.

Typical Course of Action

S#	Actor Action	System Response
1	Driver marks 'Arrived' at pickup	System notifies passenger
2	Driver marks 'Start Ride' after boarding	System begins billing and safety monitoring
3	Driver marks 'End Ride' at destination	System stops billing and triggers payment summary

Alternate Course of Action

S#	Actor Action	System Response
1	Driver ends ride early	System flags for Admin review and prompts for reason

3. 1.21 View Earnings & Fairness Score

Identifier	UC-21
Purpose	Financial transparency including equitable distribution metrics.
Priority	Medium
Pre-conditions	Logged in.
Post-conditions	Driver informed of work-life balance and pay.

Typical Course of Action

S#	Actor Action	System Response
1	Driver opens 'Earnings' tab	System aggregates daily/weekly revenue data
2		System calculates 'Fairness Score' (Idle vs Working time)
3		System displays earnings charts and score

Alternate Course of Action

S#	Actor Action	System Response
1	Database timeout	System displays "Service temporarily unavailable"

3. 1.22 Rate Passenger

Identifier	UC-22
Purpose	Providing reciprocal feedback on passenger behavior.
Priority	Medium
Pre-conditions	Ride ended.
Post-conditions	Reciprocal accountability maintained.

Typical Course of Action

S#	Actor Action	System Response
1	Driver selects 1-5 star rating	System displays optional comment box
2	Driver submits feedback	System updates passenger's average rating

Alternate Course of Action

S#	Actor Action	System Response
1	Rating out of range	System displays error and prompts for valid selection

Part 3: Admin Use Cases (23-30)

3. 1.23 Admin Secure Login

Identifier	UC-23
Purpose	Restricted access to the platform command center.
Priority	High
Pre-conditions	Admin credentials assigned.
Post-conditions	Secure admin session started.

Typical Course of Action

S#	Actor Action	System Response
1	Admin enters credentials	System validates against secure auth-server
2		System grants access to Monitoring Dashboard

Alternate Course of Action

S#	Actor Action	System Response
1	Login from unrecognized IP	System triggers secondary MFA (planned) or flags the login

3. 1.24 Real-time Live Monitoring Map

Identifier	UC-24
Purpose	Overview of all active operations across Lahore.
Priority	High
Pre-conditions	Logged in.
Post-conditions	Visual situational awareness for Admin.

Typical Course of Action

S#	Actor Action	System Response
1	Admin opens 'Live Map'	System fetches all active ride locations
2		System renders dynamic icons using Socket.io
3		System refreshes icons every 5 seconds

Alternate Course of Action

S#	Actor Action	System Response
1	Socket.io disconnection	System attempts reconnect and displays "Offline" badge

3. 1.25 Safety Sentinel Alert Management

Identifier	UC-25
Purpose	Centralized handling of automated safety escalations.
Priority	High
Pre-conditions	Anomaly detected (UC-26) or SOS triggered (UC-12).
Post-conditions	Alert incident handled or resolved.

Typical Course of Action

S#	Actor Action	System Response
1		System flashes red Critical Alert on dashboard
2	Admin clicks alert	System centers map on the incident location
3		System displays live stream of Passenger/Driver data
4	Admin clicks 'Resolve' after protocol	System logs the resolution steps and clears alert

Alternate Course of Action

S#	Actor Action	System Response
1	Admin fails to respond in 2 minutes	System auto-escalates to senior security team

3. 1.26 Route Deviation Monitoring (Auto)

Identifier	UC-26
Purpose	Automated background logic for path safety.
Priority	High
Pre-conditions	Ride is currently in 'Active' state.
Post-conditions	Autonomous safety monitoring performed.

Typical Course of Action

S#	Actor Action	System Response
1		System calculates distance from planned route polyline
2		System detects deviation > 500m
3		System triggers 'Are you okay?' Prompt (UC-11)

Alternate Course of Action

S#	Actor Action	System Response
1	Minor deviation (< 500m)	System logs local telemetry but does not alert

111

3. 1.27 Manage User Verification & Suspend

Identifier	UC-27
Purpose	Moderation of accounts to ensure platform integrity.
Priority	Medium
Pre-conditions	Logged in.
Post-conditions	User base remains verified and safe.

Typical Course of Action

S#	Actor Action	System Response
1	Admin reviews 'Pending Approval' queue	System displays registration documents

2	Admin approves or suspends user	System updates User status and sends notification

Alternate Course of Action

S#	Actor Action	System Response
1	Documents found to be fraudulent	System permanently bans CNIC and notifies law enforcement

3. 1.28 Generate AI Demand Heatmaps

Identifier	UC-28
Purpose	Business intelligence for optimizing driver distribution.
Priority	Medium
Pre-conditions	Logged in.
Post-conditions	Data-driven operational insight.

Typical Course of Action

S#	Actor Action	System Response
1	Admin selects 'Heatmap' tool	System sends parameters to AI Service
2		AI Service performs K-Means clustering on demand spikes
3		System renders color-coded density map

Alternate Course of Action

S#	Actor Action	System Response
1	Insufficient data for clustering	System displays "No heatmap available for this period"

3. 1.29 Analyze Feedback Sentiment Analytics

Identifier	UC-29
Purpose	Automated screening of thousands of reviews.
Priority	Medium

Pre-conditions	Logged in.
Post-conditions	Quality assurance at scale.

Typical Course of Action

S#	Actor Action	System Response
1	Admin opens 'Sentiment Analytics' dashboard	System fetches AI-processed sentiment scores
2		System displays trends and flags low-sentiment items

Alternate Course of Action

S#	Actor Action	System Response
1	Sudden drop in sentiment	System triggers automated alert to Admin

3. 1.30 System Reports & Export

Identifier	UC-30
Purpose	Generating statutory and performance reports.
Priority	Low
Pre-conditions	Logged in.
Post-conditions	Audit-ready documentation.

Typical Course of Action

S#	Actor Action	System Response
1	Admin selects date range and data type	System generates report preview
2	Admin clicks 'Export'	System generates and downloads PDF/CSV file

Alternate Course of Action

S#	Actor Action	System Response

1	Large report generation	System background-processes and notifies via email

3.3 Requirements Analysis and Modeling

This section includes the analysis models: entity-relationship diagram, abstract class diagram, sequence diagram. As per the project plan, these detailed visual models are documented in the Phase 2 specification. The functional requirements listed above serve as the textual detailed analysis required for this phase.

4. Nonfunctional Requirements

4.1 Performance Requirements

Response Latency: The system is engineered to provide near-real-time responses. Key API endpoints (e.g., login, ride status) must respond within 500ms for 95% of requests under normal load.

Pink Pass Processing: The entire biometric verification process, from video upload to liveness confirmation, must not exceed 15 seconds to minimize user friction.

Driver Matching: The matching algorithm must parse available drivers and return a ranked list within 3 seconds.

Concurrency: The backend infrastructure is scaled to support at least 1000 concurrent active users and handle 100 simultaneous ride sessions without degradation.

4.2 Safety Requirements

Continuous Monitoring: The Safety Sentinel service must maintain an active monitoring loop for every ongoing ride, checking GPS coordinates against the planned route at 5-second intervals.

Alert Delivery: In the event of a detected anomaly (deviation >500m), the system must successfully dispatch an SMS alert to emergency contacts and a Socket.io event to the admin dashboard within 30 seconds.

Data Integrity: All safety-critical data, including route logs and alert timestamps, must be immutable and stored for post-incident auditing.

4.3 Security Requirements

Authentication: All access to the system APIs requires a valid JSON Web Token (JWT), which expires after 24 hours. Passwords must be hashed using bcrypt with a minimum work factor of 10.

Encryption: All sensitive Personally Identifiable Information (PII), specifically CNIC numbers and biometric templates, must be encrypted at rest using AES-256 standard.

Transport Security: All data in transit between the mobile apps and the server must be secured using HTTPS/TLS 1.3 encryption to prevent man-in-the-middle attacks.

4.4 Additional Software Quality Attributes

Usability: The mobile applications shall adhere to Material Design (Android) and Human Interface Guidelines (iOS) to ensure intuitive navigation.

Reliability: The system targets a 99% uptime availability during operational hours (6 AM to 12 AM). Critical safety services are designed with redundancy to achieve 99.9% reliability.

Maintainability: The codebase shall follow standard style guides (ESLint for JS, PEP 8 for Python) to facilitate future updates. API documentation shall be maintained via Swagger.

5. Other Requirements

Legal Compliance: The system shall incorporate features to maximize alignment with local transport regulations, including digital logs of all trips.

Localization: While the initial interface is English, the architecture shall support internationalization (i18n) to allow for Urdu language support in future updates.

Hardware Compatibility: The application must remain functional on mid-range smartphones common in the target market, optimizing resource usage to prevent overheating or excessive battery drain.

6. Revised Project Plan

The SAFORA project adheres to an Agile Scrum methodology, broken down into 2-week sprints. This iterative approach allows for continuous feedback and refinement of features based on testing results.

Full 22-Week Development Roadmap:

Weeks 1–2: Phase 1 – Project Setup & Documentation

Creation of Project Proposal and SRS documentation.

Development environment setup and architecture planning.

Weeks 3–5: Phase 2 – Backend Core Development

Implementation of MongoDB schemas and Express.js server.

Development of core Authentication (JWT) and User Management APIs.

Weeks 6–8: Phase 3 – AI Microservice Development

Development of Pink Pass (Liveness Detection) and AI Pricing Models.

Implementation of the Driver Matching algorithm.

Weeks 9–12: Phase 4 – Mobile Application Development

Building the cross-platform Passenger and Driver applications (React Native).

UI/UX implementation for ride booking and dashboard.

Weeks 13–15: Phase 5 – Safety Sentinel Integration

Integration of real-time GPS tracking and Safety Sentinel logic.

Implementation of SOS triggers and automated SMS alerts (Twilio).

Weeks 16–17: Phase 6 – Admin Dashboard Implementation

Development of the web-based monitoring interface (React.js).

Integration of live maps, heatmaps, and user moderation tools.

Weeks 18–19: Phase 7 – AI Model Training & Refinement

Dataset collection and training of the CNN models.

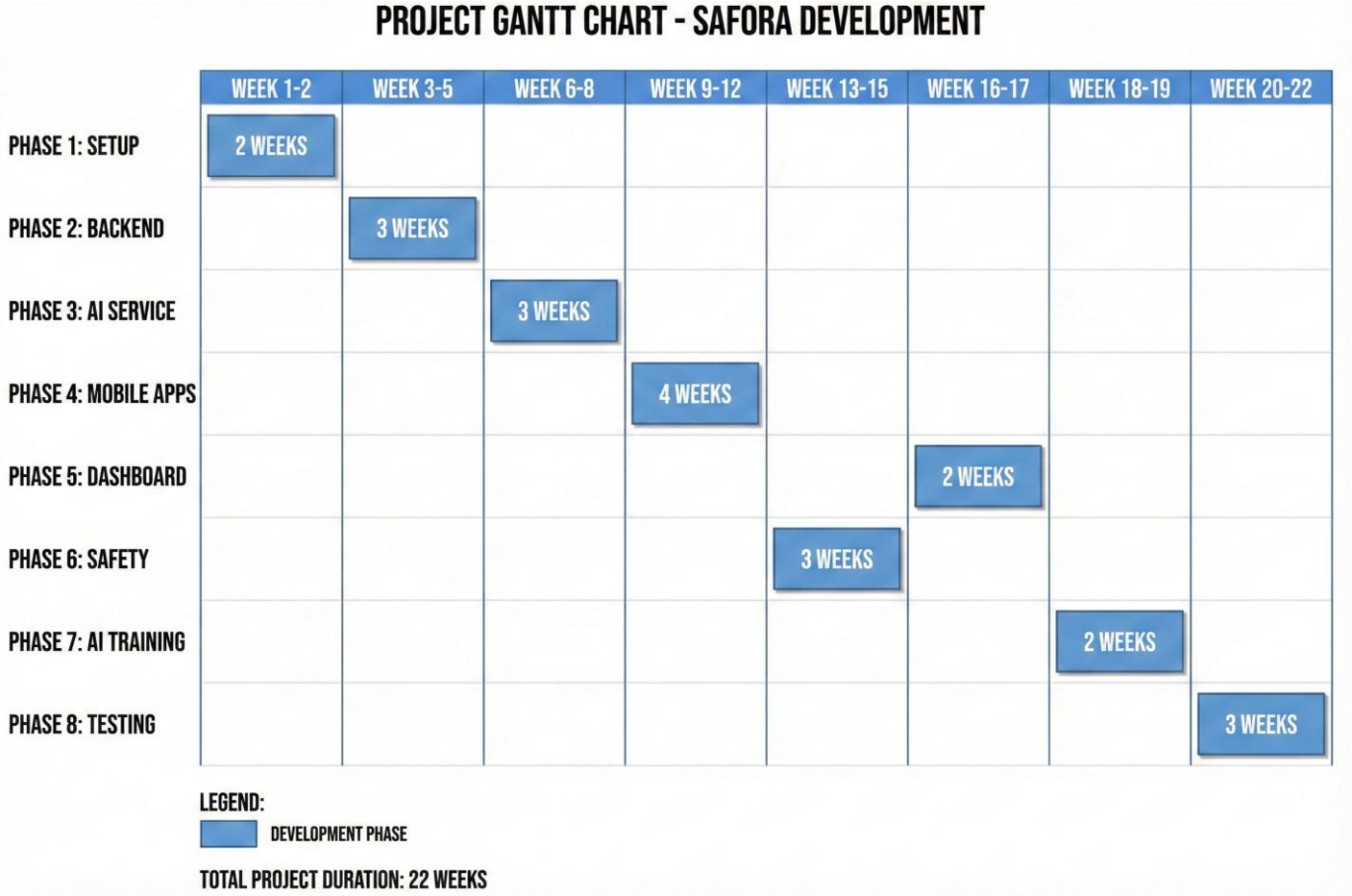
Fine-tuning of the Fairness Scoring and Sentiment Analysis modules.

Weeks 20–22: Phase 8 – Testing, QA & Deployment

Extensive Unit, Integration, and User Acceptance Testing (UAT).

Final production deployment and project handover.

This iterative approach allows for continuous feedback and refinement of features based on testing results.



7. References

[1] IEEE Std 830-1998, "IEEE Recommended Practice for Software Requirements Specifications," Institute of Electrical and Electronics Engineers, 1998.

[2] IEEE Std 29148-2018, "ISO/IEC/IEEE International Standard - Systems and software engineering -- Requirements engineering," IEEE, 2018.

[3] I. Sommerville, Software Engineering, 10th ed., Pearson Education, 2015.

[4] S. Freund, N. Hawelka, and B. Gegov, "Safety and Security Issues in App-Based On-Demand Economy: An Analysis of User Reviews for Ride-Sharing Services," Journal of the Association for Information Systems, vol. 21, no. 4, pp. 1044-1073, 2020.

[5] N. Agatz, A. Erera, M. Savelsbergh, and X. Wang, "Dynamic Ride-Sharing: A Simulation Study in Metro Atlanta," Transportation Research Part B: Methodological, vol. 45, no. 9, pp. 1450-1464, 2011.

[6] D. Goel and P. Jain, "Mobile Application Development: React Native vs Native Android and iOS," International Journal of Computer Science and Mobile Computing, vol. 9, no. 4, pp. 30-35, 2020.

[7] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), May 2015.

[8] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd ed., O'Reilly Media, 2019.

[9] K. Chodorow, MongoDB: The Definitive Guide, 3rd ed., O'Reilly Media, 2019.

[10] G. Rauch, "Socket.io: Real-time application framework for Node.js," [Online]. Available: https://socket.io. [Accessed: 14-Feb-2026].

[11] A. Anjos and S. Marcel, "Counter-Measures to Photo Attacks in Face Recognition: A Public Database and a Baseline," 2011 International Joint Conference on Biometrics (IJCB), Washington, DC, 2011, pp. 1-7.

Appendix A: Glossary

The following terms are defined to ensure a consistent understanding of this SRS document:

Term/Acronym	Definition
AI (Artificial Intelligence)	Simulation of human intelligence processes by machines, used here for decision making in driver matching and pricing.
API (Application Programming Interface)	A set of definitions and protocols for building and integrating application software.
CNN (Convolutional Neural Network)	A class of deep neural networks, most commonly applied to analyzing visual imagery, used for Pink Pass verification.
CNIC	Computerized National Identity Card, utilized for robust identity verification in Pakistan.
EAR (Eye Aspect Ratio)	A scalar quantity computed from eye landmarks to detect blinking in liveness detection.
GPS (Global Positioning System)	Satellite-based navigation system used for tracking vehicle location.
JWT (JSON Web Token)	An open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object.
Pink Pass	A proprietary feature of SAFORA offering biometrically verified, women-only rides.
Safety Sentinel	The automated background service responsible for monitoring active rides for route deviations.
Socket.io	A library that enables real-time, bidirectional and event-based communication between the browser and the server.